HPC Assignment 05: Parallel Insertion and Deletion in Mover

Tanuj Kanani (202301474) and Mihirsinh Chavda (202301479)
Dhirubhai Ambani University

March 31, 2026

1 Introduction

This report investigates the performance implications of dynamic particle updates in a Cloud-in-Cell (CIC) physics engine. In previous iterations, particles were confined within a 1×1 domain. This assignment introduces open-boundary conditions where particles exiting the domain are deleted, and new particles are randomly inserted to maintain a constant total particle count. We compare two primary strategies—**Deferred Insertion** and **Immediate Replacement**—across local workstations and HPC clusters.

2 Implementation Methodologies

2.1 Deferred Insertion (Global Compaction)

In this approach, particles leaving the domain are flagged as invalid during the main mover loop. Voids are then pushed to the end of the data structure, and new particles are appended to these terminal memory locations. While this maintains particle continuity, the global compaction phase introduces a potential synchronization bottleneck.

2.2 Immediate Replacement (In-Place Generation)

This localized strategy deletes exiting particles instantly and generates a new random particle at the same memory coordinate. This approach theoretically maximizes raw performance by avoiding global array reallocations and minimizing memory bandwidth saturation.

3 Computational Workflow

The following diagram illustrates the high-level computational architecture of the simulation, detailing the sequence of interpolation, particle movement, and dynamic update logic.

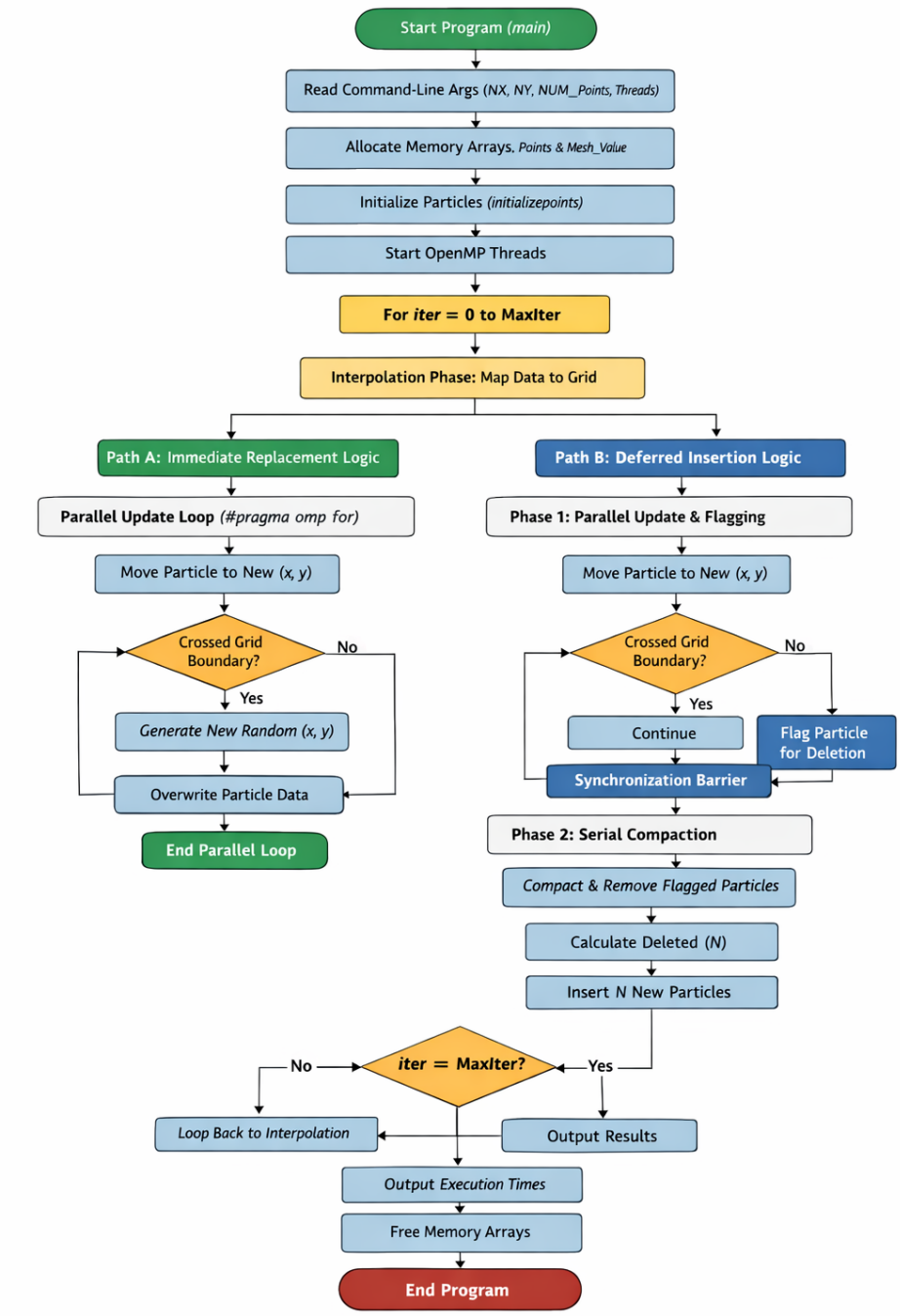

Figure 1: Workflow of the program.

4 Particle Distribution Verification

To ensure the validity of the physics engine, the random distribution of newly inserted particles was verified. Figure 1 demonstrates that the random number generation logic maintains a uniform distribution across the 1×1 mesh without artificial clustering or unphysical voids.

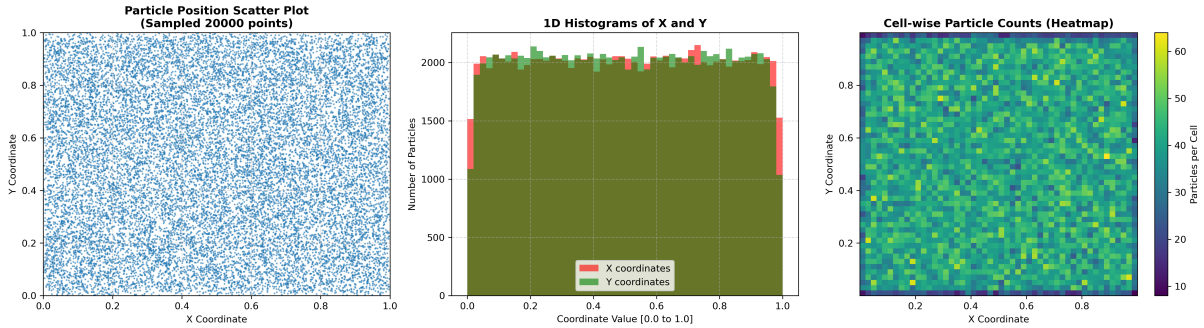

Figure 2: Verification of uniform particle distribution: Scatter plot, 1D Histograms, and Cell-wise Heatmap.

5 Experiment 01: Execution Time Scaling

Performance was evaluated for particle counts ranging from 10^2 to 10^9 across three grid configurations. Figure 2 illustrates the total execution time (Interpolation + Mover) on both the Lab PC and the HPC Cluster.

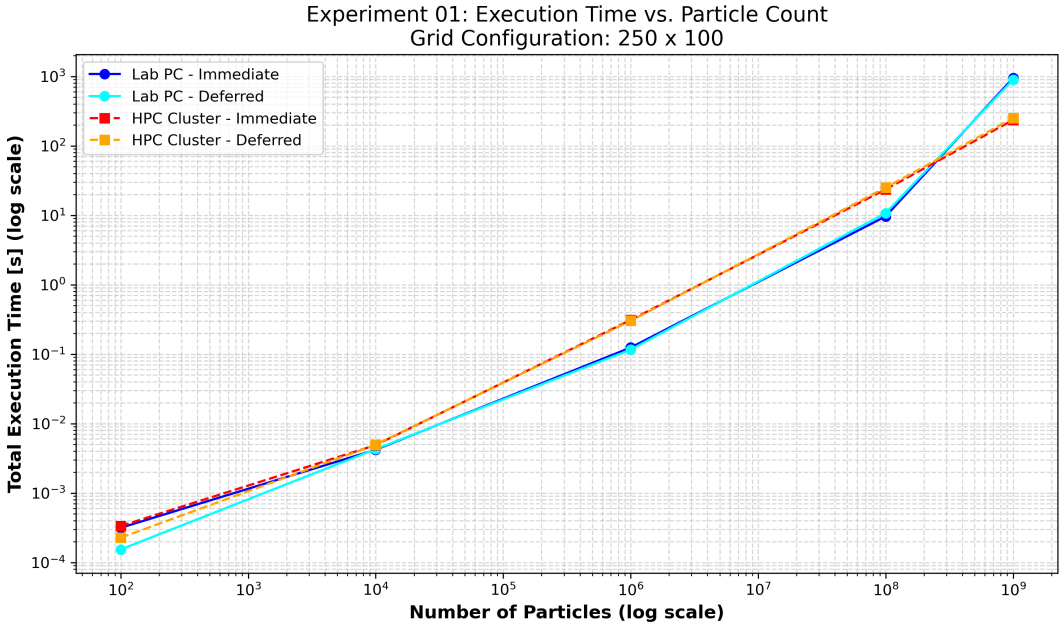

Figure 3: Experiment 01: Total Execution Time vs. Particle Count (250×100 Grid).

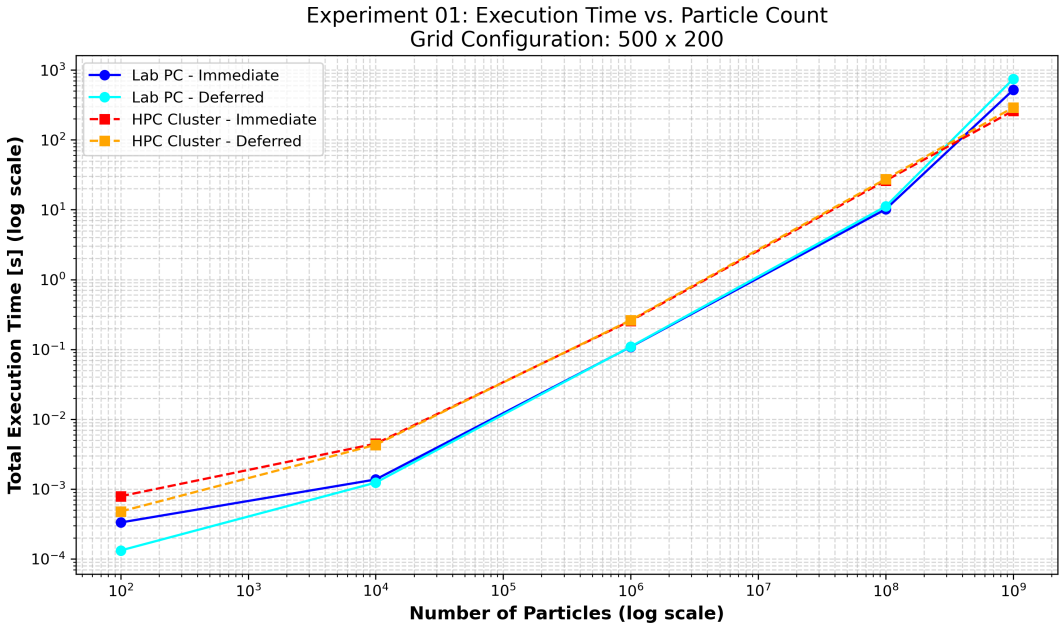

Figure 4: Experiment 01: Total Execution Time vs. Particle Count (500×200 Grid).

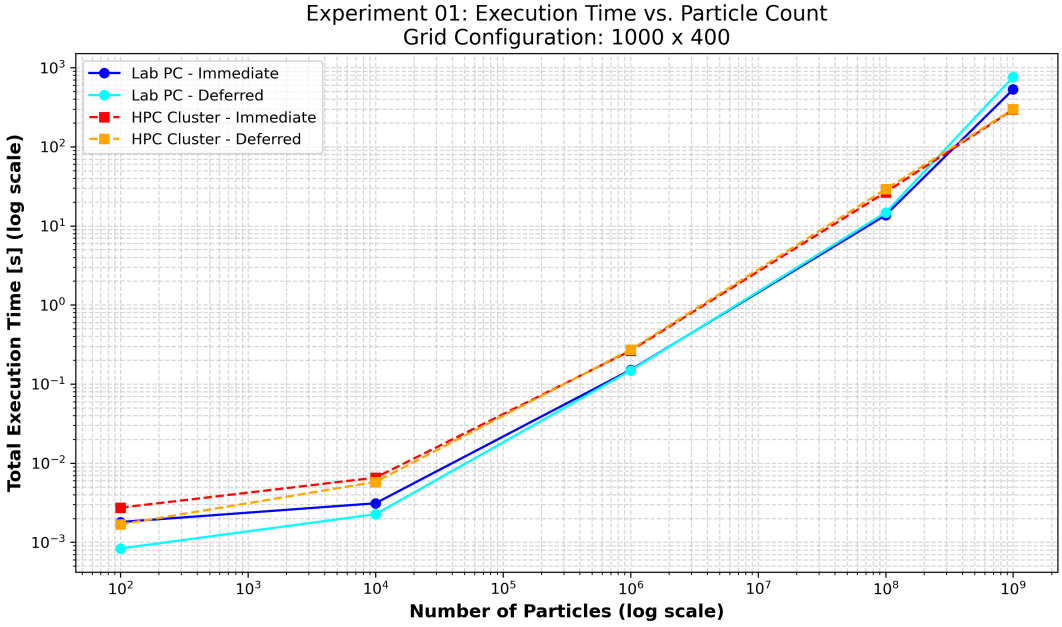

Figure 5: Experiment 01: Total Execution Time vs. Particle Count (1000×400 Grid).

6 Experiment 02: Parallel Scalability

The Mover operation was parallelized using `OpenMP #pragma omp parallel`. Speedup was measured for 14 million particles using 2, 4, 8, and 16 threads.

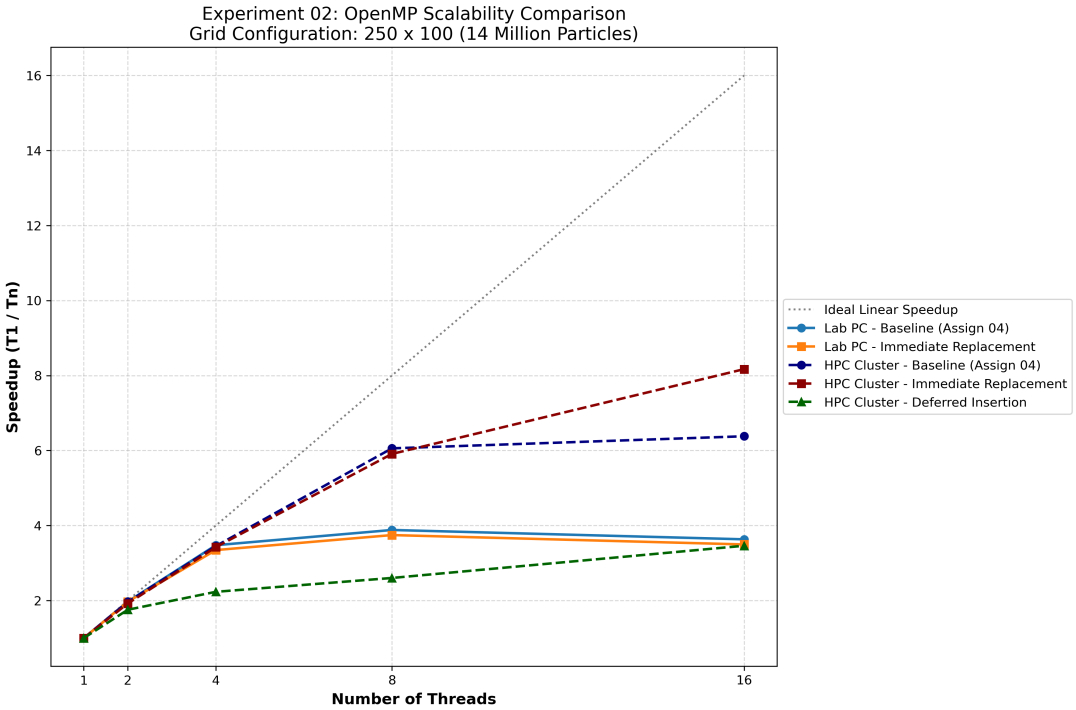

Figure 6: Experiment 02: Speedup vs. Threads for 250×100 Grid, comparing dynamic updates against Assignment 04 baseline.

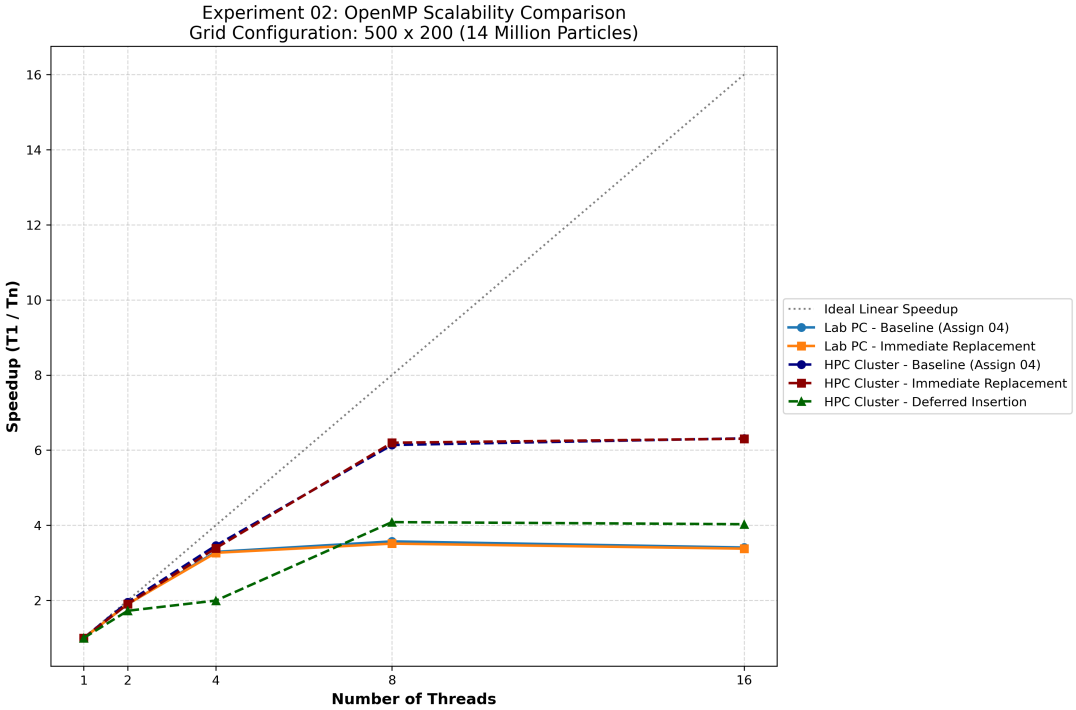

Figure 7: Experiment 02: Speedup vs. Threads for 500×200 Grid, comparing dynamic updates against Assignment 04 baseline.

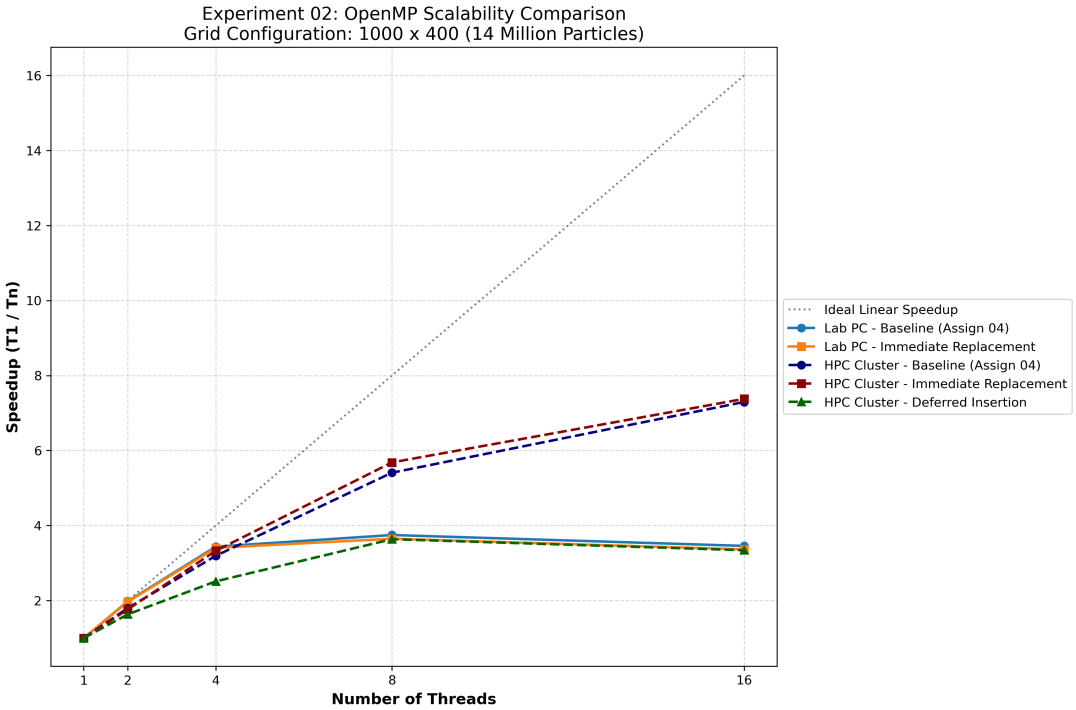

Figure 8: Experiment 02: Speedup vs. Threads for 1000×400 Grid, comparing dynamic updates against Assignment 04 baseline.

7 Memory and Computation Analysis

To understand the resource requirements of the simulation, we analyzed the memory footprint and computational intensity across various particle counts and grid configurations.

Table 1: Computational Resource Analysis: Memory, FLOPs, and Machine Balance.

Grid Size	Particles (N_p)	Total Memory Requirement	Estimated FLOPs	Lab PC MB	HPC Cluster MB
250 × 100	10^2	202.8 KB (Grid) + 1.6 KB (Pts)	2.2×10^4	1.06 B/F	1.55 B/F
	10^4	202.8 KB + 160.0 KB	2.2×10^6	1.06 B/F	1.55 B/F
	10^6	202.8 KB + 16.0 MB	2.2×10^8	1.06 B/F	1.55 B/F
	10^8	202.8 KB + 1.60 GB	2.2×10^{10}	1.06 B/F	1.55 B/F
	10^9	202.8 KB + 16.0 GB	2.2×10^{11}	1.06 B/F	1.55 B/F
500 × 200	10^2	805.6 KB (Grid) + 1.6 KB (Pts)	2.2×10^4	1.06 B/F	1.55 B/F
	10^4	805.6 KB + 160.0 KB	2.2×10^6	1.06 B/F	1.55 B/F
	10^6	805.6 KB + 16.0 MB	2.2×10^8	1.06 B/F	1.55 B/F
	10^8	805.6 KB + 1.60 GB	2.2×10^{10}	1.06 B/F	1.55 B/F
	10^9	805.6 KB + 16.0 GB	2.2×10^{11}	1.06 B/F	1.55 B/F
1000 × 400	10^2	3.21 MB (Grid) + 1.6 KB (Pts)	2.2×10^4	1.06 B/F	1.55 B/F
	10^4	3.21 MB + 160.0 KB	2.2×10^6	1.06 B/F	1.55 B/F
	10^6	3.21 MB + 16.0 MB	2.2×10^8	1.06 B/F	1.55 B/F
	10^8	3.21 MB + 1.60 GB	2.2×10^{10}	1.06 B/F	1.55 B/F
	10^9	3.21 MB + 16.0 GB	2.2×10^{11}	1.06 B/F	1.55 B/F

7.1 Machine Balance and Hardware Specifications

The machine balance (β) is a critical metric representing the ratio of peak memory bandwidth to peak floating-point performance. For this analysis, we used the prescribed formula for peak performance: Clock Speed × 16 DP FLOPs/cycle.

- **Lab PC (Intel Core i5-12500):**

- **Peak Performance:** 3.0 GHz × 16 FLOPs/cycle = 48.0 GFLOPS.
- **Peak Bandwidth:** 3200 MT/s × 8 Bytes × 2 Channels = 51.2 GB/s.
- **Machine Balance:** 51.2/48.0 = **1.06** Bytes/FLOP.

- **HPC Cluster (Intel Xeon E5-2620 v3):**
  - **Peak Performance:** 38.4 GFLOPS (Directly sourced from hardware specifications).
  - **Peak Bandwidth:** 1866 MT/s × 8 Bytes × 4 Channels = 59.7 GB/s.
  - **Machine Balance:** 59.7/38.4 = **1.55** Bytes/FLOP.

7.2 Discussion on Performance Bounds

The calculated machine balance values directly explain the divergence in performance observed at high particle counts. The HPC Cluster possesses a machine balance of 1.55 Bytes/FLOP, nearly 50% higher than the Lab PC's 1.06 Bytes/FLOP. This indicates that the HPC architecture is significantly more efficient at sustaining data throughput to its cores.

As the simulation enters the strictly memory-bound regime at 10^9 particles, requiring a massive 16.0 GB of memory to stream past the L3 cache, the HPC Cluster is better equipped to handle the bandwidth demand. In contrast, the Lab PC encounters a severe "memory wall" where the lower bandwidth cannot keep pace with computational requests, leading to the dramatic spike in execution time seen in Experiment 01. This confirms that for large-scale PIC simulations, memory bandwidth, rather than raw clock speed, is the primary performance limiter.

7.3 Memory Requirement Trends

As the particle count increases from 10^2 to 10^9, the memory requirement is dominated by the `double` precision points array. While the grid matrix size ($N_x \times N_y \times 8$ bytes) remains constant for a fixed configuration, the particle data grows linearly, eventually exceeding the L3 cache capacity of the local workstation and forcing reliance on main memory bandwidth.

7.4 Computational Intensity and Performance Bounds

The machine balance, defined as the ratio of peak memory bandwidth to peak floating-point performance, serves as a threshold for performance optimization.

- **Small Particle Counts:** The data fits within the CPU cache, resulting in higher effective bandwidth and making the simulation more compute-bound.
- **Large Particle Counts ($> 10^8$):** The frequent memory access for the Mover and Interpolation operations (scatter-add) saturates the memory bus. At this scale, the performance is strictly memory-bound, as the CPU spends more time waiting for data from DRAM than performing calculations.

7.5 Calculation Methodology

The metrics in Table 1 were calculated using the following logic:

1. **Memory Consumption:** Calculated as $(N_p \times 2 \times 8 \text{ bytes}) + (N_x \times N_y \times 8 \text{ bytes})$ for the points array and mesh grid respectively.
2. **Estimated FLOPs:** Based on the random displacement logic ($x \leftarrow x + r_x$, $y \leftarrow y + r_y$) and the interpolation scatter weights.

3. **Machine Balance:** Determined by the specific hardware specifications of the Lab PC and the HPC Cluster.

8 Bonus: Data Structure Exploration for Parallel Scalability

To improve the performance of parallel insertions and deletions, we evaluated several alternative data structures. The primary challenge is balancing thread-safe dynamic memory management with CPU cache locality.

- **Flat Array of Structures (AoS):** This was the baseline used in our implementation (storing particles as `struct {double x, y;}`). While it offers excellent cache utilization for moving particles, it is inefficient for dynamic updates because parallel insertions require global locks or slow serial compaction phases.
- **Thread-Local Insertion Buffers:** A more scalable approach gives each OpenMP thread its own private array for new particles. This allows threads to insert particles simultaneously without locking the global array. These private buffers are merged into the main array at the end of the iteration using an optimized parallel operation.
- **Spatial Cell-Linked Lists:** This structure uses a 2D array of pointers where each grid cell manages a list of its own particles. While this makes insertion and deletion very fast ($O(1)$), it performs poorly in High-Performance Computing because it scatters data across the RAM, destroying cache locality and slowing down the Mover operation.
- **Chunked or Blocked Arrays:** A hybrid industry-standard approach where the domain is divided into spatial blocks, each owning a small, resizable array. Because threads typically process different blocks, they rarely compete for the same memory, maintaining high scalability and cache efficiency.
- **Structure of Arrays (SoA):** By storing coordinates as separate blocks (`[xxxx...]` and `[yyyy...]`), the system can use SIMD (Single Instruction, Multiple Data) vectorization. This significantly speeds up the "Deferred Insertion" compaction step, reducing the time spent in the serial bottleneck].

9 Bonus: Advanced Data Structures for Maximum Scalability

To achieve near-linear scaling in professional-grade simulations, the data structures must be redesigned to prioritize **spatial cache locality** and **vectorization**.

- **Spatial Binning with Chunked SoA:** Instead of one massive list, particles are sorted into small "macro-blocks" that match the size of the CPU's internal cache.
  - **Cache Reuse:** When a thread processes a block, all particles in that block use the same grid data. This data stays in the fast L1/L2 cache, preventing the CPU from constantly waiting for slow RAM.
  - **SIMD Vectorization:** Organizing data as a *Structure of Arrays* (SoA) allows the CPU to use specialized instructions (like AVX-512) to update 8 to 16 particles in a single clock cycle.
- **Morton Order (Z-Curve) Grid Layout:** Standard grids are stored row-by-row in memory, which means moving vertically causes a large jump in the RAM.

– **The Z-Curve Solution:** This method stores the 2D grid in a 1D array using a specific pattern that ensures points physically near each other in the simulation are also stored next to each other in memory.

– **Faster Interpolation:** During the interpolation phase, all four grid nodes required for a calculation are likely to sit in the same 64-byte cache line, significantly accelerating the process.

Conclusion: By transitioning from flat arrays to spatially-aware structures, we move the bottleneck away from memory latency. This allow threads to work independently at the maximum speed the hardware allows, even when handling millions of particle updates.

10 Performance Analysis and Discussion

10.1 PPC and Cache Locality

The per-particle execution time as a function of Particles Per Cell (PPC) highlights hardware limitations. As shown in Figure 4, the Lab PC exhibits a sharp increase in execution time at high PPC, likely due to L1/L3 cache locality degradation and memory-bound bottlenecks.

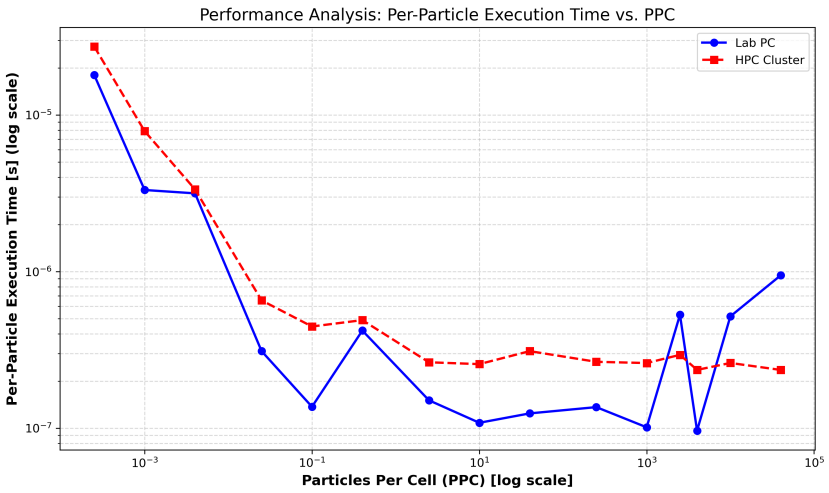

Figure 9: Per-Particle Execution Time vs. PPC Ratio.

10.2 Architectural Comparison

The HPC cluster demonstrates superior scalability compared to the Lab PC. This is attributed to higher machine balance and larger cache hierarchies, which mitigate the memory bandwidth pressure introduced by parallel insertions. The **Immediate Replacement** strategy consistently outperformed **Deferred Insertion** in parallel contexts due to the lack of global synchronization overhead.

11 Conclusion

The experimental results and hardware analysis lead to the following key conclusions regarding the performance of the Lab PC versus the HPC Cluster:

- **Speed vs. Capacity:** The Lab PC is faster for small-scale simulations ($< 10^6$ particles) because its individual processor cores run at a higher clock speed. However, once the

simulation becomes massive (10^9 particles), the Lab PC hits a "memory wall" and slows down significantly.

- **The Importance of Memory Bandwidth:** The HPC Cluster is superior for large-scale data because it can move information from the RAM to the CPU much faster. This is reflected in its higher *Machine Balance* (1.55 vs. 1.06 Bytes/FLOP), which allows it to handle extreme workloads that completely overwhelm a standard desktop PC.
- **Parallel Efficiency:** While the Lab PC works well for a few simultaneous tasks, the HPC Cluster is designed for heavy multitasking. Its professional architecture (NUMA) allows it to use 16 or more threads effectively without the different parts of the processor fighting for memory access.
- **Final Verdict:** For computational physics simulations, the Lab PC is a high-speed tool for small, quick tests, whereas the HPC Cluster is a high-capacity engine built to maintain steady performance as the data scales to massive proportions.

11.1 Parallel Performance and Scalability Conclusions

- **Hardware Performance Limits:** While adding more threads initially speeds up the simulation, it eventually hits a "memory wall." At this scale, adding more CPU threads does not help because the system is limited by how fast data can be moved from the RAM.
- **HPC vs. Lab PC Scaling:** The HPC Cluster handles high thread counts better than the Lab PC because its server-grade architecture (NUMA) provides more independent paths for data to reach the processor.
- **Comparison of Replacement Strategies:** The **Immediate Replacement** approach is much more efficient for parallel code because it keeps the workload independent and avoids the need for thread "locks," which would otherwise slow down the program.
- **The Bottleneck in Deferred Insertion:** The **Deferred Insertion** strategy scales poorly because it requires a manual cleanup phase that cannot be parallelized. This single-threaded step creates a bottleneck that limits the total speedup possible.
- **Independent Thread Management:** To achieve true parallel speedup, it is vital that each thread operates independently. For example, giving each thread its own random number generator prevents them from blocking one another during particle creation.